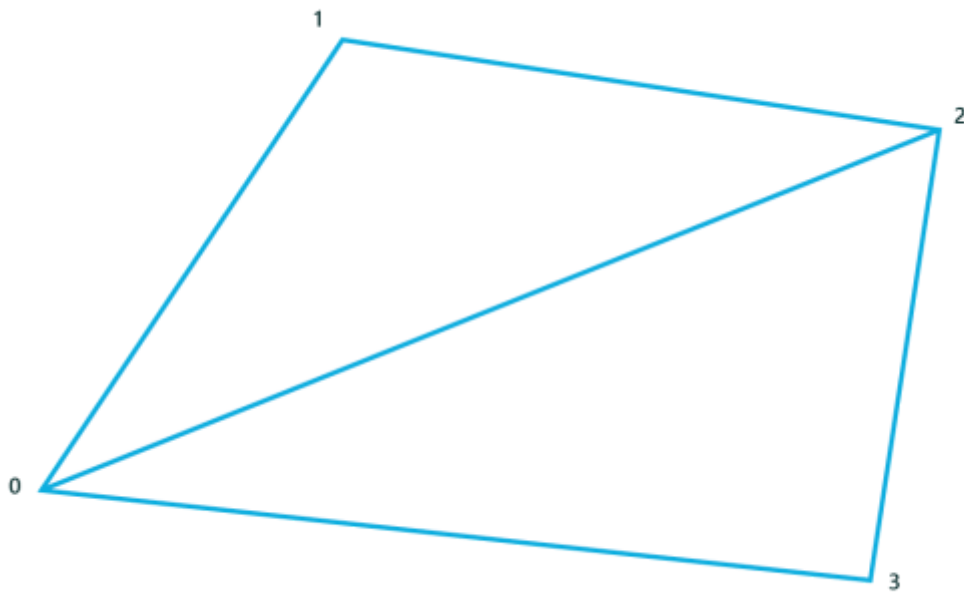


In our example cube, you have 8 vertices, which create 6 quads for the sides. You split the quads into triangles, for a total of 12 triangles that use our 8 vertices. At 3 vertices per triangle, you have 36 entries in our index buffer. In our example, this index pattern is known as a triangle list, and you indicate it to Direct3D as a **D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST** when you set the primitive topology.

This is probably the most inefficient way to list indices, as there are many redundancies when triangles share points and sides. For example, when a triangle shares a side in a rhombus shape, you list 6 indices for the four vertices, like this:



- Triangle 1: [0, 1, 2]
- Triangle 2: [0, 2, 3]

In a strip or fan topology, you order the vertices in a way that eliminates many redundant sides during traversal (such as the side from index 0 to index 2 in the image.) For large meshes, this dramatically reduces the number of times the vertex shader is run, and improves performance significantly. However, we'll keep it simple and stick with the triangle list.

Declare the indices for the vertex buffer as a simple triangle list topology.

C++

```
unsigned short cubeIndices[] =  
{  
    0, 1, 2,  
    0, 2, 3,  
  
    4, 5, 6,
```

```
4, 6, 7,  
  
3, 2, 5,  
3, 5, 4,  
  
2, 1, 6,  
2, 6, 5,  
  
1, 7, 6,  
1, 0, 7,  
  
0, 3, 4,  
0, 4, 7 };
```

Thirty six index elements in the buffer is very redundant when you only have 8 vertices! If you choose to eliminate some of the redundancies and use a different vertex list type, such as a strip or a fan, you must specify that type when you provide a specific `D3D11_PRIMITIVE_TOPOLOGY` value to the `ID3D11DeviceContext::IASetPrimitiveTopology` method.

For more information about different index list techniques, see [Primitive Topologies](#).

Step 5: Create a constant buffer for your transformation matrices

Before you can start processing vertices, you need to provide the transformation matrices that will be applied (multiplied) to each vertex when it runs. For most 3-D games, there are three of them:

- The 4x4 matrix that transforms from the object (model) coordinate system to the overall world coordinate system.
- The 4x4 matrix that transforms from the world coordinate system to the camera (view) coordinate system.
- The 4x4 matrix that transforms from the camera coordinate system to the 2-D view projection coordinate system.

These matrices are passed to the shader in a *constant buffer*. A constant buffer is a region of memory that remains constant throughout the execution of the next pass of the shader pipeline, and which can be directly accessed by the shaders from your HLSL code. You define each constant buffer two times: first in your game's C++ code, and (at least) one time in the C-like HLSL syntax for your shader code. The two declarations must directly correspond in terms of types and data alignment. It's easy to introduce hard to find errors when the shader uses the HLSL declaration to interpret data declared in C++, and the types don't match or the alignment of data is off!

Constant buffers don't get changed by the HLSL. You can change them when your game updates specific data. Often, game devs create 4 classes of constant buffers: one type for updates per frame; one type for updates per model/object; one type for updates per game state refresh; and one type for data that never changes through the lifetime of the game.

In this example, we just have one that never changes: the DirectX::XMFLOAT4X4 data for the three matrices.

Note The example code presented here uses column-major matrices. You can use row-major matrices instead by using the **row_major** keyword in HLSL, and ensuring your source matrix data is also row-major. DirectXMath uses row-major matrices and can be used directly with HLSL matrices defined with the **row_major** keyword.

Declare and create a constant buffer for the three matrices you use to transform each vertex.

C++

```
struct ConstantBuffer
{
    DirectX::XMFLOAT4X4 model;
    DirectX::XMFLOAT4X4 view;
    DirectX::XMFLOAT4X4 projection;
};
ComPtr<ID3D11Buffer> m_constantBuffer;
ConstantBuffer m_constantBufferData;

// ...

// Create a constant buffer for passing model, view, and projection matrices
// to the vertex shader. This allows us to rotate the cube and apply
// a perspective projection to it.

D3D11_BUFFER_DESC constantBufferDesc = {0};
constantBufferDesc.ByteWidth = sizeof(m_constantBufferData);
constantBufferDesc.Usage = D3D11_USAGE_DEFAULT;
constantBufferDesc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
constantBufferDesc.CPUAccessFlags = 0;
constantBufferDesc.MiscFlags = 0;
constantBufferDesc.StructureByteStride = 0;
m_d3dDevice->CreateBuffer(
    &constantBufferDesc,
    nullptr,
    &m_constantBuffer
);
```

```

m_constantBufferData.model = DirectX::XMFLOAT4X4( // Identity matrix, since
you are not animating the object
    1.0f, 0.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f, 1.0f);

);
// Specify the view (camera) transform corresponding to a camera position of
// X = 0, Y = 1, Z = 2.

m_constantBufferData.view = DirectX::XMFLOAT4X4(
    -1.00000000f, 0.00000000f, 0.00000000f, 0.00000000f,
    0.00000000f, 0.89442718f, 0.44721359f, 0.00000000f,
    0.00000000f, 0.44721359f, -0.89442718f, -2.23606800f,
    0.00000000f, 0.00000000f, 0.00000000f, 1.00000000f);

```

Note You usually declare the projection matrix when you set up device specific resources, because the results of multiplication with it must match the current 2-D viewport size parameters (which often correspond with the pixel height and width of the display). If those change, you must scale the x- and y-coordinate values accordingly.

C++

```

// Finally, update the constant buffer perspective projection parameters
// to account for the size of the application window. In this sample,
// the parameters are fixed to a 70-degree field of view, with a depth
// range of 0.01 to 100.

float xScale = 1.42814801f;
float yScale = 1.42814801f;
if (backBufferDesc.Width > backBufferDesc.Height)
{
    xScale = yScale *
        static_cast<float>(backBufferDesc.Height) /
        static_cast<float>(backBufferDesc.Width);
}
else
{
    yScale = xScale *
        static_cast<float>(backBufferDesc.Width) /
        static_cast<float>(backBufferDesc.Height);
}
m_constantBufferData.projection = DirectX::XMFLOAT4X4(
    xScale, 0.0f, 0.0f, 0.0f,
    0.0f, yScale, 0.0f, 0.0f,
    0.0f, 0.0f, -1.0f, -0.01f,

```

```
0.0f, 0.0f, -1.0f, 0.0f
);
```

While you're here, set the vertex and index buffers on the [ID3D11DeviceContext](#), plus the topology you're using.

C++

```
// Set the vertex and index buffers, and specify the way they define
geometry.
UINT stride = sizeof(SimpleCubeVertex);
UINT offset = 0;
m_d3dDeviceContext->IASetVertexBuffers(
    0,
    1,
    vertexBuffer.GetAddressOf(),
    &stride,
    &offset);

m_d3dDeviceContext->IASetIndexBuffer(
    indexBuffer.Get(),
    DXGI_FORMAT_R16_UINT,
    0);

m_d3dDeviceContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
```

All right! Input assembly complete. Everything's in place for rendering. Let's get that vertex shader going.

Step 6: Process the mesh with the vertex shader

Now that you have a vertex buffer with the vertices that define your mesh, and the index buffer that defines the order in which the vertices are processed, you send them to the vertex shader. The vertex shader code, expressed as compiled high-level shader language, runs one time for each vertex in the vertex buffer, allowing you to perform your per-vertex transforms. The final result is typically a 2-D projection.

(Did you load your vertex shader? If not, review [How to load resources in your DirectX game.](#))

Here, you create the vertex shader...

syntax

```
// Set the vertex and pixel shader stage state.
m_d3dDeviceContext->VSSetShader(
```

```
vertexShader.Get(),  
nullptr,  
0);
```

...and set the constant buffers.

syntax

```
m_d3dDeviceContext->VSSetConstantBuffers(  
    0,  
    1,  
    m_constantBuffer.GetAddressOf());
```

Here's the vertex shader code that handles the transformation from object coordinates to world coordinates and then to the 2-D view projection coordinate system. You also apply some simple per-vertex lighting to make things pretty. This goes in your vertex shader's HLSL file (SimpleVertexShader.hlsl, in this example).

syntax

```
cbuffer simpleConstantBuffer : register( b0 )  
{  
    matrix model;  
    matrix view;  
    matrix projection;  
};  
  
struct VertexShaderInput  
{  
    DirectX::XMFLOAT3 pos : POSITION;  
    DirectX::XMFLOAT3 color : COLOR;  
};  
  
struct PixelShaderInput  
{  
    float4 pos : SV_POSITION;  
    float4 color : COLOR;  
};  
  
PixelShaderInput SimpleVertexShader(VertexShaderInput input)  
{  
    PixelShaderInput vertexShaderOutput;  
    float4 pos = float4(input.pos, 1.0f);  
  
    // Transform the vertex position into projection space.  
    pos = mul(pos, model);  
    pos = mul(pos, view);  
    pos = mul(pos, projection);  
    vertexShaderOutput.pos = pos;  
  
    // Pass the vertex color through to the pixel shader.
```

```

    vertexShaderOutput.color = float4(input.color, 1.0f);

    return vertexShaderOutput;
}

```

See that **cbuffer** at the top? That's the HLSL analogue to the same constant buffer we declared in our C++ code previously. And the **VertexShaderInputstruct**? Why, that looks just like your input layout and vertex data declaration! It's important that the constant buffer and vertex data declarations in your C++ code match the declarations in your HLSL code—and that includes signs, types, and data alignment.

PixelShaderInput specifies the layout of the data that is returned by the vertex shader's main function. When you finish processing a vertex, you'll return a vertex position in the 2-D projection space and a color used for per-vertex lighting. The graphics card uses data output by the shader to calculate the "fragments" (possible pixels) that must be colored when the pixel shader is run in the next stage of the pipeline.

Step 7: Passing the mesh through the pixel shader

Typically, at this stage in the graphics pipeline, you perform per-pixel operations on the visible projected surfaces of your objects. (People like textures.) For the purposes of sample, though, you simply pass it through this stage.

First, let's create an instance of the pixel shader. The pixel shader runs for every pixel in the 2-D projection of your scene, assigning a color to that pixel. In this case, we pass the color for the pixel returned by the vertex shader straight through.

Set the pixel shader.

syntax

```
m_d3dDeviceContext->PSSetShader( pixelShader.Get(), nullptr, 0 );
```

Define a passthrough pixel shader in HLSL.

syntax

```

struct PixelShaderInput
{
    float4 pos : SV_POSITION;
};

float4 SimplePixelShader(PixelShaderInput input) : SV_TARGET
{
    // Draw the entire triangle yellow.
}

```

```
    return float4(1.0f, 1.0f, 0.0f, 1.0f);  
}
```

Put this code in an HLSL file separate from the vertex shader HLSL (such as SimplePixelShader.hlsl). This code is run one time for every visible pixel in your viewport (an in-memory representation of the portion of the screen you are drawing to), which, in this case, maps to the entire screen. Now, your graphics pipeline is completely defined!

Step 8: Rasterizing and displaying the mesh

Let's run the pipeline. This is easy: call [ID3D11DeviceContext::DrawIndexed](#).

Draw that cube!

```
C++  
  
// Draw the cube.  
m_d3dDeviceContext->DrawIndexed( ARRAYSIZE(cubeIndices), 0, 0 );
```

Inside the graphics card, each vertex is processed in the order specified in your index buffer. After your code has executed the vertex shader and the 2-D fragments are defined, the pixel shader is invoked and the triangles colored.

Now, put the cube on the screen.

Present that frame buffer to the display.

```
C++  
  
// Present the rendered image to the window. Because the maximum frame  
// latency is set to 1,  
// the render loop is generally throttled to the screen refresh rate,  
// typically around  
// 60 Hz, by sleeping the app on Present until the screen is refreshed.  
  
m_swapChain->Present(1, 0);
```

And you're done! For a scene full of models, use multiple vertex and index buffers, and you might even have different shaders for different model types. Remember that each model has its own coordinate system, and you need to transform them to the shared world coordinate system using the matrices you defined in the constant buffer.

Remarks

This topic covers creating and displaying simple geometry that you create yourself. For more info about loading more complex geometry from a file and converting it to the sample-specific vertex buffer object (.vbo) format, see [How to load resources in your DirectX game](#).

Related topics

- [How to load resources in your DirectX game](#)

Load resources in your DirectX game

Article • 10/20/2022

Most games, at some point, load resources and assets (such as shaders, textures, predefined meshes or other graphics data) from local storage or some other data stream. Here, we walk you through a high-level view of what you must consider when loading these files to use in your DirectX C/C++ Universal Windows Platform (UWP) game.

For example, the meshes for polygonal objects in your game might have been created with another tool, and exported to a specific format. The same is true for textures, and more so: while a flat, uncompressed bitmap can be commonly written by most tools and understood by most graphics APIs, it can be extremely inefficient for use in your game. Here, we guide you through the basic steps for loading three different types of graphic resources for use with DirectX3D: meshes (models), textures (bitmaps), and compiled shader objects.

What you need to know

Technologies

- Parallel Patterns Library (ppltasks.h)

Prerequisites

- Understand the basic Windows Runtime
- Understand asynchronous tasks
- Understand the basic concepts of 3-D graphics programming.

This sample also includes three code files for resource loading and management. You'll encounter the code objects defined in these files throughout this topic.

- BasicLoader.h/.cpp
- BasicReaderWriter.h/.cpp
- DDSTextureLoader.h/.cpp

The complete code for these samples can be found in the following links.

Topic	Description
-------	-------------

Topic	Description
Complete code for BasicLoader	Complete code for a class and methods that convert and load graphics mesh objects into memory.
Complete code for BasicReaderWriter	Complete code for a class and methods for reading and writing binary data files in general. Used by the BasicLoader class.
Complete code for DDSTextureLoader	Complete code for a class and method that loads a DDS texture from memory.

Instructions

Asynchronous loading

Asynchronous loading is handled using the **task** template from the Parallel Patterns Library (PPL). A **task** contains a method call followed by a lambda that processes the results of the async call after it completes, and usually follows the format of:

```
task<generic return type>(async code to execute).then((parameters for lambda){  
    lambda code contents });
```

Tasks can be chained together using the **.then()** syntax, so that when one operation completes, another async operation that depends on the results of the prior operation can be run. In this way, you can load, convert, and manage complex assets on separate threads in a way that appears almost invisible to the player.

For more details, read [Asynchronous programming in C++](#).

Now, let's look at the basic structure for declaring and creating an async file loading method, **ReadDataAsync**.

C++

```
#include <ppltasks.h>  
  
// ...  
concurrency::task<Platform::Array<byte>^> ReadDataAsync(  
    _In_ Platform::String^ filename);  
  
// ...
```

```

using concurrency;

task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename
)
{
    return task<StorageFile^>(m_location->GetFileAsync(filename)).then([=]
    (StorageFile^ file)
    {
        return FileIO::ReadBufferAsync(file);
    }).then([=](IBuffer^ buffer)
    {
        auto fileData = ref new Platform::Array<byte>(buffer->Length);
        DataReader::FromBuffer(buffer)->ReadBytes(fileData);
        return fileData;
    });
}

```

In this code, when your code calls the **ReadDataAsync** method defined above, a task is created to read a buffer from the file system. Once it completes, a chained task takes the buffer and streams the bytes from that buffer into an array using the static **DataReader** type.

```

C++

m_basicReaderWriter = ref new BasicReaderWriter();

// ...
return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ bytecode)
{
    // Perform some operation with the data when the async load completes.
});

```

Here's the call you make to **ReadDataAsync**. When it completes, your code receives an array of bytes read from the provided file. Since **ReadDataAsync** itself is defined as a task, you can use a lambda to perform a specific operation when the byte array is returned, such as passing that byte data to a DirectX function that can use it.

If your game is sufficiently simple, load your resources with a method like this when the user starts the game. You can do this before you start the main game loop from some point in the call sequence of your **IFrameworkView::Run** implementation. Again, you call your resource loading methods asynchronously so the game can start quicker and so the player doesn't have to wait until the loading completes before engaging in early interactions.

However, you don't want to start the game proper until all of the async loading has completed! Create some method for signaling when loading is complete, such as a specific field, and use the lambdas on your loading method(s) to set that signal when finished. Check the variable before starting any components that use those loaded resources.

Here's an example using the async methods defined in BasicLoader.cpp to load shaders, a mesh, and a texture when the game starts up. Notice that it sets a specific field on the game object, **m_loadingComplete**, when all of the loading methods finish.

C++

```
void ResourceLoading::CreateDeviceResources()
{
    // DirectXBase is a common sample class that implements a basic view
    // provider.

    DirectXBase::CreateDeviceResources();

    // ...

    // This flag will keep track of whether or not all application
    // resources have been loaded. Until all resources are loaded,
    // only the sample overlay will be drawn on the screen.
    m_loadingComplete = false;

    // Create a BasicLoader, and use it to asynchronously load all
    // application resources. When an output value becomes non-null,
    // this indicates that the asynchronous operation has completed.
    BasicLoader^ loader = ref new BasicLoader(m_d3dDevice.Get());

    auto loadVertexShaderTask = loader->LoadShaderAsync(
        "SimpleVertexShader.cso",
        nullptr,
        0,
        &m_vertexShader,
        &m_inputLayout
    );

    auto loadPixelShaderTask = loader->LoadShaderAsync(
        "SimplePixelShader.cso",
        &m_pixelShader
    );

    auto loadTextureTask = loader->LoadTextureAsync(
        "reftexture.dds",
        nullptr,
        &m_textureSRV
    );

    auto loadMeshTask = loader->LoadMeshAsync(
        "refmesh.vbo",
```

```

        &m_vertexBuffer,
        &m_indexBuffer,
        nullptr,
        &m_indexCount
    );

    // The && operator can be used to create a single task that represents
    // a group of multiple tasks. The new task's completed handler will only
    // be called once all associated tasks have completed. In this case, the
    // new task represents a task to load various assets from the package.
    (loadVertexShaderTask && loadPixelShaderTask && loadTextureTask &&
loadMeshTask).then([=]()
    {
        m_loadingComplete = true;
    });

    // Create constant buffers and other graphics device-specific resources
    here.
}

```

Note that the tasks have been aggregated using the && operator such that the lambda that sets the loading complete flag is triggered only when all of the tasks complete. Note that if you have multiple flags, you have the possibility of race conditions. For example, if the lambda sets two flags sequentially to the same value, another thread may only see the first flag set if it examines them before the second flag is set.

You've seen how to load resource files asynchronously. Synchronous file loads are much simpler, and you can find examples of them in [Complete code for BasicReaderWriter](#) and [Complete code for BasicLoader](#).

Of course, different resource and asset types often require additional processing or conversion before they are ready to be used in your graphics pipeline. Let's take a look at three specific types of resources: meshes, textures, and shaders.

Loading meshes

Meshes are vertex data, either generated procedurally by code within your game or exported to a file from another app (like 3DStudio MAX or Alias WaveFront) or tool. These meshes represent the models in your game, from simple primitives like cubes and spheres to cars and houses and characters. They often contain color and animation data, as well, depending on their format. We'll focus on meshes that contain only vertex data.

To load a mesh correctly, you must know the format of the data in the file for the mesh. Our simple **BasicReaderWriter** type above simply reads the data in as a byte stream; it doesn't know that the byte data represents a mesh, much less a specific mesh format as

exported by another application! You must perform the conversion as you bring the mesh data into memory.

(You should always try to package asset data in a format that's as close to the internal representation as possible. Doing so will reduce resource utilization and save time.)

Let's get the byte data from the mesh's file. The format in the example assumes that the file is a sample-specific format suffixed with `.vbo`. (Again, this format is not the same as OpenGL's VBO format.) Each vertex itself maps to the **BasicVertex** type, which is a struct defined in the code for the `obj2vbo` converter tool. The layout of the vertex data in the `.vbo` file looks like this:

- The first 32 bits (4 bytes) of the data stream contain the number of vertices (`numVertices`) in the mesh, represented as a `uint32` value.
- The next 32 bits (4 bytes) of the data stream contain the number of indices in the mesh (`numIndices`), represented as a `uint32` value.
- After that, the subsequent (`numVertices * sizeof(BasicVertex)`) bits contain the vertex data.
- The last (`numIndices * 16`) bits of data contain the index data, represented as a sequence of `uint16` values.

The point is this: know the bit-level layout of the mesh data you have loaded. Also, be sure you are consistent with endian-ness. All Windows 8 platforms are little-endian.

In the example, you call a method, `CreateMesh`, from the **LoadMeshAsync** method to perform this bit-level interpretation.

C++

```
task<void> BasicLoader::LoadMeshAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ meshData)
    {
        CreateMesh(
            meshData->Data,
            vertexBuffer,
            indexBuffer,
            vertexCount,
            indexCount,
            filename
        );
    });
}
```

```
});
}
```

CreateMesh interprets the byte data loaded from the file, and creates a vertex buffer and an index buffer for the mesh by passing the vertex and index lists, respectively, to [ID3D11Device::CreateBuffer](#) and specifying either `D3D11_BIND_VERTEX_BUFFER` or `D3D11_BIND_INDEX_BUFFER`. Here's the code used in **BasicLoader**:

C++

```
void BasicLoader::CreateMesh(
    _In_ byte* meshData,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount,
    _In_opt_ Platform::String^ debugName
)
{
    // The first 4 bytes of the BasicMesh format define the number of
    vertices in the mesh.
    uint32 numVertices = *reinterpret_cast<uint32*>(meshData);

    // The following 4 bytes define the number of indices in the mesh.
    uint32 numIndices = *reinterpret_cast<uint32*>(meshData +
    sizeof(uint32));

    // The next segment of the BasicMesh format contains the vertices of the
    mesh.
    BasicVertex* vertices = reinterpret_cast<BasicVertex*>(meshData +
    sizeof(uint32) * 2);

    // The last segment of the BasicMesh format contains the indices of the
    mesh.
    uint16* indices = reinterpret_cast<uint16*>(meshData + sizeof(uint32) *
    2 + sizeof(BasicVertex) * numVertices);

    // Create the vertex and index buffers with the mesh data.

    D3D11_SUBRESOURCE_DATA vertexBufferData = {0};
    vertexBufferData.pSysMem = vertices;
    vertexBufferData.SysMemPitch = 0;
    vertexBufferData.SysMemSlicePitch = 0;
    CD3D11_BUFFER_DESC vertexBufferDesc(numVertices * sizeof(BasicVertex),
    D3D11_BIND_VERTEX_BUFFER);

    m_d3dDevice->CreateBuffer(
        &vertexBufferDesc,
        &vertexBufferData,
        vertexBuffer
    );
}
```



```

D3D11_SUBRESOURCE_DATA indexBufferData = {0};
indexBufferData.pSysMem = indices;
indexBufferData.SysMemPitch = 0;
indexBufferData.SysMemSlicePitch = 0;
CD3D11_BUFFER_DESC indexBufferDesc(numIndices * sizeof(uint16),
D3D11_BIND_INDEX_BUFFER);

m_d3dDevice->CreateBuffer(
    &indexBufferDesc,
    &indexBufferData,
    indexBuffer
);

if (vertexCount != nullptr)
{
    *vertexCount = numVertices;
}
if (indexCount != nullptr)
{
    *indexCount = numIndices;
}
}

```

You typically create a vertex/index buffer pair for every mesh you use in your game. Where and when you load the meshes is up to you. If you have a lot of meshes, you may only want to load some from the disk at specific points in the game, such as during specific, pre-defined loading states. For large meshes, like terrain data, you can stream the vertices from a cache, but that is a more complex procedure and not in the scope of this topic.

Again, know your vertex data format! There are many, many ways to represent vertex data across the tools used to create models. There are also many different ways to represent the input layout of the vertex data to Direct3D, such as triangle lists and strips. For more information about vertex data, read [Introduction to Buffers in Direct3D 11](#) and [Primitives](#).

Next, let's look at loading textures.

Loading textures

The most common asset in a game—and the one that comprises most of the files on disk and in memory—are textures. Like meshes, textures can come in a variety of formats, and you convert them to a format that Direct3D can use when you load them. Textures also come in a wide variety of types and are used to create different effects. MIP levels for textures can be used to improve the look and performance of distance objects; dirt and light maps are used to layer effects and detail atop a base texture; and normal maps are used in per-pixel lighting calculations. In a modern game, a typical

scene can potentially have thousands of individual textures, and your code must effectively manage them all!

Also like meshes, there are a number of specific formats that are used to make memory usage for efficient. Since textures can easily consume a large portion of the GPU (and system) memory, they are often compressed in some fashion. You aren't required to use compression on your game's textures, and you can use any compression/decompression algorithm(s) you want as long as you provide the Direct3D shaders with data in a format it can understand (like a [Texture2D](#) bitmap).

Direct3D provides support for the DXT texture compression algorithms, although every DXT format may not be supported in the player's graphics hardware. DDS files contain DXT textures (and other texture compression formats as well), and are suffixed with .dds.

A DDS file is a binary file that contains the following information:

- A DWORD (magic number) containing the four character code value 'DDS ' (0x20534444).
- A description of the data in the file.

The data is described with a header description using [DDS_HEADER](#); the pixel format is defined using [DDS_PIXELFORMAT](#). Note that the **DDS_HEADER** and **DDS_PIXELFORMAT** structures replace the deprecated DDSURFACEDESC2, DDSCAPS2 and DDPIXELFORMAT DirectDraw 7 structures. **DDS_HEADER** is the binary equivalent of DDSURFACEDESC2 and DDSCAPS2. **DDS_PIXELFORMAT** is the binary equivalent of DDPIXELFORMAT.

C++

```
DWORD          dwMagic;  
DDS_HEADER     header;
```

If the value of **dwFlags** in [DDS_PIXELFORMAT](#) is set to DDPF_FOURCC and **dwFourCC** is set to "DX10" an additional [DDS_HEADER_DXT10](#) structure will be present to accommodate texture arrays or DXGI formats that cannot be expressed as an RGB pixel format such as floating point formats, sRGB formats etc. When the **DDS_HEADER_DXT10** structure is present, the entire data description will look like this.

C++

```
DWORD          dwMagic;  
DDS_HEADER     header;  
DDS_HEADER_DXT10 header10;
```

- A pointer to an array of bytes that contains the main surface data.

C++

```
BYTE bdata[]
```

- A pointer to an array of bytes that contains the remaining surfaces such as; mipmap levels, faces in a cube map, depths in a volume texture. Follow these links for more information about the DDS file layout for a: [texture](#), a [cube map](#), or a [volume texture](#).

C++

```
BYTE bdata2[]
```

Many tools export to the DDS format. If you don't have a tool to export your texture to this format, consider creating one. For more detail on the DDS format and how to work with it in your code, read [Programming Guide for DDS](#). In our example, we'll use DDS.

As with other resource types, you read the data from a file as a stream of bytes. Once your loading task completes, the lambda call runs code (the **CreateTexture** method) to process the stream of bytes into a format that Direct3D can use.

C++

```
task<void> BasicLoader::LoadTextureAsync(
    _In_ Platform::String^ filename,
    _Out_opt_ ID3D11Texture2D** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ textureData)
    {
        CreateTexture(
            GetExtension(filename) == "dds",
            textureData->Data,
            textureData->Length,
            texture,
            textureView,
            filename
        );
    });
}
```

In the previous snippet, the lambda checks to see if the filename has an extension of "dds". If it does, you assume that it is a DDS texture. If not, well, use the Windows Imaging Component (WIC) APIs to discover the format and decode the data as a bitmap. Either way, the result is a [Texture2D](#) bitmap (or an error).

C++

```
void BasicLoader::CreateTexture(
    _In_ bool decodeAsDDS,
    _In_reads_bytes_(dataSize) byte* data,
    _In_ uint32 dataSize,
    _Out_opt_ ID3D11Texture2D** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView,
    _In_opt_ Platform::String^ debugName
)
{
    ComPtr<ID3D11ShaderResourceView> shaderResourceView;
    ComPtr<ID3D11Texture2D> texture2D;

    if (decodeAsDDS)
    {
        ComPtr<ID3D11Resource> resource;

        if (textureView == nullptr)
        {
            CreateDDSTextureFromMemory(
                m_d3dDevice.Get(),
                data,
                dataSize,
                &resource,
                nullptr
            );
        }
        else
        {
            CreateDDSTextureFromMemory(
                m_d3dDevice.Get(),
                data,
                dataSize,
                &resource,
                &shaderResourceView
            );
        }

        resource.As(&texture2D);
    }
    else
    {
        if (m_wicFactory.Get() == nullptr)
        {
            // A WIC factory object is required in order to load texture
            // assets stored in non-DDS formats. If BasicLoader was not
            // initialized with one, create one as needed.
        }
    }
}
```

```

        CoCreateInstance(
            CLSID_WICImagingFactory,
            nullptr,
            CLSCTX_INPROC_SERVER,
            IID_PPV_ARGS(&m_wicFactory));
    }

    ComPtr<IWICStream> stream;
    m_wicFactory->CreateStream(&stream);

    stream->InitializeFromMemory(
        data,
        dataSize);

    ComPtr<IWICBitmapDecoder> bitmapDecoder;
    m_wicFactory->CreateDecoderFromStream(
        stream.Get(),
        nullptr,
        WICDecodeMetadataCacheOnDemand,
        &bitmapDecoder);

    ComPtr<IWICBitmapFrameDecode> bitmapFrame;
    bitmapDecoder->GetFrame(0, &bitmapFrame);

    ComPtr<IWICFormatConverter> formatConverter;
    m_wicFactory->CreateFormatConverter(&formatConverter);

    formatConverter->Initialize(
        bitmapFrame.Get(),
        GUID_WICPixelFormat32bppPBGRA,
        WICBitmapDitherTypeNone,
        nullptr,
        0.0,
        WICBitmapPaletteTypeCustom);

    uint32 width;
    uint32 height;
    bitmapFrame->GetSize(&width, &height);

    std::unique_ptr<byte[]> bitmapPixels(new byte[width * height * 4]);
    formatConverter->CopyPixels(
        nullptr,
        width * 4,
        width * height * 4,
        bitmapPixels.get());

    D3D11_SUBRESOURCE_DATA initialData;
    ZeroMemory(&initialData, sizeof(initialData));
    initialData.pSysMem = bitmapPixels.get();
    initialData.SysMemPitch = width * 4;
    initialData.SysMemSlicePitch = 0;

    CD3D11_TEXTURE2D_DESC textureDesc(
        DXGI_FORMAT_B8G8R8A8_UNORM,
        width,

```

```

        height,
        1,
        1
    );

    m_d3dDevice->CreateTexture2D(
        &textureDesc,
        &initialData,
        &texture2D);

    if (textureView != nullptr)
    {
        CD3D11_SHADER_RESOURCE_VIEW_DESC shaderResourceViewDesc(
            texture2D.Get(),
            D3D11_SRV_DIMENSION_TEXTURE2D
        );

        m_d3dDevice->CreateShaderResourceView(
            texture2D.Get(),
            &shaderResourceViewDesc,
            &shaderResourceView);
    }
}

if (texture != nullptr)
{
    *texture = texture2D.Detach();
}
if (textureView != nullptr)
{
    *textureView = shaderResourceView.Detach();
}
}

```

When this code completes, you have a **Texture2D** in memory, loaded from an image file. As with meshes, you probably have a lot of them in your game and in any given scene. Consider creating caches for regularly accessed textures per-scene or per-level, rather than loading them all when the game or level starts.

(The **CreateDDSTextureFromMemory** method called in the above sample can be explored in full in [Complete code for DDSTextureLoader](#).)

Also, individual textures or texture "skins" may map to specific mesh polygons or surfaces. This mapping data is usually exported by the tool an artist or designer used to create the model and the textures. Make sure that you capture this information as well when you load the exported data, as you will use it map the correct textures to the corresponding surfaces when you perform fragment shading.

Loading shaders

Shaders are compiled High Level Shader Language (HLSL) files that are loaded into memory and invoked at specific stages of the graphics pipeline. The most common and essential shaders are the vertex and pixel shaders, which process the individual vertices of your mesh and the pixels in the scene's viewport(s), respectively. The HLSL code is executed to transform the geometry, apply lighting effects and textures, and perform post-processing on the rendered scene.

A Direct3D game can have a number of different shaders, each one compiled into a separate CSO (Compiled Shader Object, .cso) file. Normally, you don't have so many that you need to load them dynamically, and in most cases, you can simply load them when the game is starting, or on a per-level basis (such as a shader for rain effects).

The code in the **BasicLoader** class provides a number of overloads for different shaders, including vertex, geometry, pixel, and hull shaders. The code below covers pixel shaders as an example. (You can review the complete code in [Complete code for BasicLoader.](#))

C++

```
concurrency::task<void> LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11PixelShader** shader
);

// ...

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11PixelShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ bytecode)
    {
        m_d3dDevice->CreatePixelShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader);
    });
}
```

In this example, you use the **BasicReaderWriter** instance (**m_basicReaderWriter**) to read in the supplied compiled shader object (.cso) file as a byte stream. Once that task completes, the lambda calls **ID3D11Device::CreatePixelShader** with the byte data loaded

from the file. Your callback must set some flag indicating that the load was successful, and your code must check this flag before running the shader.

Vertex shaders are bit more complex. For a vertex shader, you also load a separate input layout that defines the vertex data. The following code can be used to asynchronously load a vertex shader along with a custom vertex input layout. Be sure that the vertex information that you load from your meshes can be correctly represented by this input layout!

Let's create the input layout before you load the vertex shader.

C++

```
void BasicLoader::CreateInputLayout(
    _In_reads_bytes_(bytecodeSize) byte* bytecode,
    _In_ uint32 bytecodeSize,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC*
    layoutDesc,
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11InputLayout** layout
)
{
    if (layoutDesc == nullptr)
    {
        // If no input layout is specified, use the BasicVertex layout.
        const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
        {
            { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
            D3D11_INPUT_PER_VERTEX_DATA, 0 },
            { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
            D3D11_INPUT_PER_VERTEX_DATA, 0 },
            { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
            D3D11_INPUT_PER_VERTEX_DATA, 0 },
        };

        m_d3dDevice->CreateInputLayout(
            basicVertexLayoutDesc,
            ARRAYSIZE(basicVertexLayoutDesc),
            bytecode,
            bytecodeSize,
            layout);
    }
    else
    {
        m_d3dDevice->CreateInputLayout(
            layoutDesc,
            layoutDescNumElements,
            bytecode,
            bytecodeSize,
            layout);
    }
}
```


In this particular layout, each vertex has the following data processed by the vertex shader:

- A 3D coordinate position (x, y, z) in the model's coordinate space, represented as a trio of 32-bit floating point values.
- A normal vector for the vertex, also represented as three 32-bit floating point values.
- A transformed 2D texture coordinate value (u, v) , represented as a pair of 32-bit floating values.

These per-vertex input elements are called [HLSL semantics](#), and they are a set of defined registers used to pass data to and from your compiled shader object. Your pipeline runs the vertex shader once for every vertex in the mesh that you've loaded. The semantics define the input to (and output from) the vertex shader as it runs, and provide this data for your per-vertex computations in your shader's HLSL code.

Now, load the vertex shader object.

C++

```
concurrency::task<void> LoadShaderAsync(
    _In_ Platform::String^ filename,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
layoutDesc[],
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11VertexShader** shader,
    _Out_opt_ ID3D11InputLayout** layout
);

// ...

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
layoutDesc[],
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11VertexShader** shader,
    _Out_opt_ ID3D11InputLayout** layout
)
{
    // This method assumes that the lifetime of input arguments may be
    shorter
    // than the duration of this task. In order to ensure accurate results,
    a
    // copy of all arguments passed by pointer must be made. The method
    then
    // ensures that the lifetime of the copied data exceeds that of the
    task.
```

```

    // Create copies of the layoutDesc array as well as the SemanticName
    strings,
    // both of which are pointers to data whose lifetimes may be shorter
    than that
    // of this method's task.
    shared_ptr<vector<D3D11_INPUT_ELEMENT_DESC>> layoutDescCopy;
    shared_ptr<vector<string>> layoutDescSemanticNamesCopy;
    if (layoutDesc != nullptr)
    {
        layoutDescCopy.reset(
            new vector<D3D11_INPUT_ELEMENT_DESC>(
                layoutDesc,
                layoutDesc + layoutDescNumElements
            )
        );

        layoutDescSemanticNamesCopy.reset(
            new vector<string>(layoutDescNumElements)
        );

        for (uint32 i = 0; i < layoutDescNumElements; i++)
        {
            layoutDescSemanticNamesCopy->at(i).assign(layoutDesc[i].SemanticName);
        }
    }

    return m_basicReaderWriter->ReadDataAsync(filename).then([=])(const
Platform::Array<byte>^ bytecode)
    {
        m_d3dDevice->CreateVertexShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader);

        if (layout != nullptr)
        {
            if (layoutDesc != nullptr)
            {
                // Reassign the SemanticName elements of the layoutDesc
                array copy to point
                // to the corresponding copied strings. Performing the
                assignment inside the
                // lambda body ensures that the lambda will take a reference
                to the shared_ptr
                // that holds the data. This will guarantee that the data
                is still valid when
                // CreateInputLayout is called.
                for (uint32 i = 0; i < layoutDescNumElements; i++)
                {
                    layoutDescCopy->at(i).SemanticName =
                    layoutDescSemanticNamesCopy->at(i).c_str();
                }
            }
        }
    }

```

```

        CreateInputLayout(
            bytecode->Data,
            bytecode->Length,
            layoutDesc == nullptr ? nullptr : layoutDescCopy->data(),
            layoutDescNumElements,
            layout);
    }
});
}

```

In this code, once you've read in the byte data for the vertex shader's CSO file, you create the vertex shader by calling [ID3D11Device::CreateVertexShader](#). After that, you create your input layout for the shader in the same lambda.

Other shader types, such as hull and geometry shaders, can also require specific configuration. Complete code for a variety of shader loading methods is provided in [Complete code for BasicLoader](#) and in the [Direct3D resource loading sample](#).

Remarks

At this point, you should understand and be able to create or modify methods for asynchronously loading common game resources and assets, such as meshes, textures, and compiled shaders.

Related topics

- [Direct3D resource loading sample](#)
- [Complete code for BasicLoader](#)
- [Complete code for BasicReaderWriter](#)
- [Complete code for DDSTextureLoader](#)

Complete code for BasicLoader

Article • 10/20/2022

Complete code for a class and methods that convert and load common graphics resources, such as meshes, textures, and various shader objects.

This topic contains these sections:

- [Technologies](#)
- [Requirements](#)
- [View the code \(C++\)](#)

Download location

This sample is not available for download.

Technologies

Programming languages - C++

Programming models - Windows Runtime

Requirements

Minimum supported client - Windows 10

Minimum supported server - Windows Server 2016 Technical Preview

View the code (C++)

BasicLoader.h

C++

```
//// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
//// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
//// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
//// PARTICULAR PURPOSE.
////
//// Copyright (c) Microsoft Corporation. All rights reserved

#pragma once
```

```

#include "BasicReaderWriter.h"

// A simple loader class that provides support for loading shaders,
// textures,
// and meshes from files on disk. Provides synchronous and asynchronous
// methods.
ref class BasicLoader
{
internal:
    BasicLoader(
        _In_ ID3D11Device* d3dDevice,
        _In_opt_ IWICImagingFactory2* wicFactory = nullptr
    );

    void LoadTexture(
        _In_ Platform::String^ filename,
        _Out_opt_ ID3D11Texture2D** texture,
        _Out_opt_ ID3D11ShaderResourceView** textureView
    );

    concurrency::task<void> LoadTextureAsync(
        _In_ Platform::String^ filename,
        _Out_opt_ ID3D11Texture2D** texture,
        _Out_opt_ ID3D11ShaderResourceView** textureView
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
layoutDesc[],
        _In_ uint32 layoutDescNumElements,
        _Out_ ID3D11VertexShader** shader,
        _Out_opt_ ID3D11InputLayout** layout
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
layoutDesc[],
        _In_ uint32 layoutDescNumElements,
        _Out_ ID3D11VertexShader** shader,
        _Out_opt_ ID3D11InputLayout** layout
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _Out_ ID3D11PixelShader** shader
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _Out_ ID3D11PixelShader** shader
    );

    void LoadShader(

```

```

        _In_ Platform::String^ filename,
        _Out_ ID3D11ComputeShader** shader
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _Out_ ID3D11ComputeShader** shader
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _Out_ ID3D11GeometryShader** shader
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _Out_ ID3D11GeometryShader** shader
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _In_reads_opt_(numEntries) const D3D11_SO_DECLARATION_ENTRY*
streamOutDeclaration,
        _In_ uint32 numEntries,
        _In_reads_opt_(numStrides) const uint32* bufferStrides,
        _In_ uint32 numStrides,
        _In_ uint32 rasterizedStream,
        _Out_ ID3D11GeometryShader** shader
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _In_reads_opt_(numEntries) const D3D11_SO_DECLARATION_ENTRY*
streamOutDeclaration,
        _In_ uint32 numEntries,
        _In_reads_opt_(numStrides) const uint32* bufferStrides,
        _In_ uint32 numStrides,
        _In_ uint32 rasterizedStream,
        _Out_ ID3D11GeometryShader** shader
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _Out_ ID3D11HullShader** shader
    );

    concurrency::task<void> LoadShaderAsync(
        _In_ Platform::String^ filename,
        _Out_ ID3D11HullShader** shader
    );

    void LoadShader(
        _In_ Platform::String^ filename,
        _Out_ ID3D11DomainShader** shader
    );

```

```

concurrency::task<void> LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11DomainShader** shader
);

void LoadMesh(
    _In_ Platform::String^ filename,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount
);

concurrency::task<void> LoadMeshAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount
);

private:
    Microsoft::WRL::ComPtr<ID3D11Device> m_d3dDevice;
    Microsoft::WRL::ComPtr<IWICImagingFactory2> m_wicFactory;
    BasicReaderWriter^ m_basicReaderWriter;

    template <class DeviceChildType>
    inline void SetDebugName(
        _In_ DeviceChildType* object,
        _In_ Platform::String^ name
    );

    Platform::String^ GetExtension(
        _In_ Platform::String^ filename
    );

    void CreateTexture(
        _In_ bool decodeAsDDS,
        _In_reads_bytes_(dataSize) byte* data,
        _In_ uint32 dataSize,
        _Out_opt_ ID3D11Texture2D** texture,
        _Out_opt_ ID3D11ShaderResourceView** textureView,
        _In_opt_ Platform::String^ debugName
    );

    void CreateInputLayout(
        _In_reads_bytes_(bytecodeSize) byte* bytecode,
        _In_ uint32 bytecodeSize,
        _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC*
layoutDesc,
        _In_ uint32 layoutDescNumElements,
        _Out_ ID3D11InputLayout** layout
    );

```

```

void CreateMesh(
    _In_ byte* meshData,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount,
    _In_opt_ Platform::String^ debugName
);
};

```

BasicLoader.cpp

C++

```

//// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
//// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
//// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
//// PARTICULAR PURPOSE.
////
//// Copyright (c) Microsoft Corporation. All rights reserved

#include "pch.h"
#include "BasicLoader.h"
#include "BasicShapes.h"
#include "DDSTextureLoader.h"
#include "DirectXSample.h"
#include <memory>

using namespace Microsoft::WRL;
using namespace Windows::Storage;
using namespace Windows::Storage::Streams;
using namespace Windows::Foundation;
using namespace Windows::ApplicationModel;
using namespace std;
using namespace concurrency;

BasicLoader::BasicLoader(
    _In_ ID3D11Device* d3dDevice,
    _In_opt_ IWICImagingFactory2* wicFactory
) :
    m_d3dDevice(d3dDevice),
    m_wicFactory(wicFactory)
{
    // Create a new BasicReaderWriter to do raw file I/O.
    m_basicReaderWriter = ref new BasicReaderWriter();
}

template <class DeviceChildType>
inline void BasicLoader::SetDebugName(
    _In_ DeviceChildType* object,
    _In_ Platform::String^ name
)

```



```

{
    #if defined(_DEBUG)
        // Only assign debug names in debug builds.

        char nameString[1024];
        int nameStringLength = WideCharToMultiByte(
            CP_ACP,
            0,
            name->Data(),
            -1,
            nameString,
            1024,
            nullptr,
            nullptr
        );

        if (nameStringLength == 0)
        {
            char defaultNameString[] = "BasicLoaderObject";
            DX::ThrowIfFailed(
                object->SetPrivateData(
                    WKPDIID_D3DDebugObjectName,
                    sizeof(defaultNameString) - 1,
                    defaultNameString
                )
            );
        }
        else
        {
            DX::ThrowIfFailed(
                object->SetPrivateData(
                    WKPDIID_D3DDebugObjectName,
                    nameStringLength - 1,
                    nameString
                )
            );
        }
    #endif
}

Platform::String^ BasicLoader::GetExtension(
    _In_ Platform::String^ filename
)
{
    int lastDotIndex = -1;
    for (int i = filename->Length() - 1; i >= 0 && lastDotIndex == -1; i--)
    {
        if (*(filename->Data() + i) == '.')
        {
            lastDotIndex = i;
        }
    }
    if (lastDotIndex != -1)
    {
        std::unique_ptr<wchar_t[]> extension(new wchar_t[filename->Length()

```

```

- lastDotIndex]);
    for (unsigned int i = 0; i < filename->Length() - lastDotIndex; i++)
    {
        extension[i] = tolower(*(filename->Data() + lastDotIndex + 1 +
i));
    }
    return ref new Platform::String(extension.get());
}
return "";
}

void BasicLoader::CreateTexture(
    _In_ bool decodeAsDDS,
    _In_reads_bytes_(dataSize) byte* data,
    _In_ uint32 dataSize,
    _Out_opt_ ID3D11Texture2D** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView,
    _In_opt_ Platform::String^ debugName
)
{
    ComPtr<ID3D11ShaderResourceView> shaderResourceView;
    ComPtr<ID3D11Texture2D> texture2D;

    if (decodeAsDDS)
    {
        ComPtr<ID3D11Resource> resource;

        if (textureView == nullptr)
        {
            CreateDDSTextureFromMemory(
                m_d3dDevice.Get(),
                data,
                dataSize,
                &resource,
                nullptr
            );
        }
        else
        {
            CreateDDSTextureFromMemory(
                m_d3dDevice.Get(),
                data,
                dataSize,
                &resource,
                &shaderResourceView
            );
        }

        DX::ThrowIfFailed(
            resource.As(&texture2D)
        );
    }
    else
    {
        if (m_wicFactory.Get() == nullptr)

```

```

{
    // A WIC factory object is required in order to load texture
    // assets stored in non-DDS formats. If BasicLoader was not
    // initialized with one, create one as needed.
    DX::ThrowIfFailed(
        CoCreateInstance(
            CLSID_WICImagingFactory,
            nullptr,
            CLSCTX_INPROC_SERVER,
            IID_PPV_ARGS(&m_wicFactory)
        )
    );
}

ComPtr<IWICStream> stream;
DX::ThrowIfFailed(
    m_wicFactory->CreateStream(&stream)
);

DX::ThrowIfFailed(
    stream->InitializeFromMemory(
        data,
        dataSize
    )
);

ComPtr<IWICBitmapDecoder> bitmapDecoder;
DX::ThrowIfFailed(
    m_wicFactory->CreateDecoderFromStream(
        stream.Get(),
        nullptr,
        WICDecodeMetadataCacheOnDemand,
        &bitmapDecoder
    )
);

ComPtr<IWICBitmapFrameDecode> bitmapFrame;
DX::ThrowIfFailed(
    bitmapDecoder->GetFrame(0, &bitmapFrame)
);

ComPtr<IWICFormatConverter> formatConverter;
DX::ThrowIfFailed(
    m_wicFactory->CreateFormatConverter(&formatConverter)
);

DX::ThrowIfFailed(
    formatConverter->Initialize(
        bitmapFrame.Get(),
        GUID_WICPixelFormat32bppPBGRA,
        WICBitmapDitherTypeNone,
        nullptr,
        0.0,
        WICBitmapPaletteTypeCustom
    )
);

```

```

    );

    uint32 width;
    uint32 height;
    DX::ThrowIfFailed(
        bitmapFrame->GetSize(&width, &height)
    );

    std::unique_ptr<byte[]> bitmapPixels(new byte[width * height * 4]);
    DX::ThrowIfFailed(
        formatConverter->CopyPixels(
            nullptr,
            width * 4,
            width * height * 4,
            bitmapPixels.get()
        )
    );

    D3D11_SUBRESOURCE_DATA initialData;
    ZeroMemory(&initialData, sizeof(initialData));
    initialData.pSysMem = bitmapPixels.get();
    initialData.SysMemPitch = width * 4;
    initialData.SysMemSlicePitch = 0;

    CD3D11_TEXTURE2D_DESC textureDesc(
        DXGI_FORMAT_B8G8R8A8_UNORM,
        width,
        height,
        1,
        1
    );

    DX::ThrowIfFailed(
        m_d3dDevice->CreateTexture2D(
            &textureDesc,
            &initialData,
            &texture2D
        )
    );

    if (textureView != nullptr)
    {
        CD3D11_SHADER_RESOURCE_VIEW_DESC shaderResourceViewDesc(
            texture2D.Get(),
            D3D11_SRV_DIMENSION_TEXTURE2D
        );

        DX::ThrowIfFailed(
            m_d3dDevice->CreateShaderResourceView(
                texture2D.Get(),
                &shaderResourceViewDesc,
                &shaderResourceView
            )
        );
    }
}

```

```

    }

    SetDebugName(texture2D.Get(), debugName);

    if (texture != nullptr)
    {
        *texture = texture2D.Detach();
    }
    if (textureView != nullptr)
    {
        *textureView = shaderResourceView.Detach();
    }
}

void BasicLoader::CreateInputLayout(
    _In_reads_bytes_(bytecodeSize) byte* bytecode,
    _In_ uint32 bytecodeSize,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC*
layoutDesc,
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11InputLayout** layout
)
{
    if (layoutDesc == nullptr)
    {
        // If no input layout is specified, use the BasicVertex layout.
        const D3D11_INPUT_ELEMENT_DESC basicVertexLayoutDesc[] =
        {
            { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
            { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
            { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
        };

        DX::ThrowIfFailed(
            m_d3dDevice->CreateInputLayout(
                basicVertexLayoutDesc,
                ARRAYSIZE(basicVertexLayoutDesc),
                bytecode,
                bytecodeSize,
                layout
            )
        );
    }
    else
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateInputLayout(
                layoutDesc,
                layoutDescNumElements,
                bytecode,
                bytecodeSize,
                layout
            )
        );
    }
}

```

```

        )
    };
}

void BasicLoader::CreateMesh(
    _In_ byte* meshData,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount,
    _In_opt_ Platform::String^ debugName
)
{
    // The first 4 bytes of the BasicMesh format define the number of
    vertices in the mesh.
    uint32 numVertices = *reinterpret_cast<uint32*>(meshData);

    // The following 4 bytes define the number of indices in the mesh.
    uint32 numIndices = *reinterpret_cast<uint32*>(meshData +
    sizeof(uint32));

    // The next segment of the BasicMesh format contains the vertices of the
    mesh.
    BasicVertex* vertices = reinterpret_cast<BasicVertex*>(meshData +
    sizeof(uint32) * 2);

    // The last segment of the BasicMesh format contains the indices of the
    mesh.
    uint16* indices = reinterpret_cast<uint16*>(meshData + sizeof(uint32) *
    2 + sizeof(BasicVertex) * numVertices);

    // Create the vertex and index buffers with the mesh data.

    D3D11_SUBRESOURCE_DATA vertexBufferData = {0};
    vertexBufferData.pSysMem = vertices;
    vertexBufferData.SysMemPitch = 0;
    vertexBufferData.SysMemSlicePitch = 0;
    CD3D11_BUFFER_DESC vertexBufferDesc(numVertices * sizeof(BasicVertex),
    D3D11_BIND_VERTEX_BUFFER);
    DX::ThrowIfFailed(
        m_d3dDevice->CreateBuffer(
            &vertexBufferDesc,
            &vertexBufferData,
            vertexBuffer
        )
    );

    D3D11_SUBRESOURCE_DATA indexBufferData = {0};
    indexBufferData.pSysMem = indices;
    indexBufferData.SysMemPitch = 0;
    indexBufferData.SysMemSlicePitch = 0;
    CD3D11_BUFFER_DESC indexBufferDesc(numIndices * sizeof(uint16),
    D3D11_BIND_INDEX_BUFFER);
    DX::ThrowIfFailed(

```

```

        m_d3dDevice->CreateBuffer(
            &indexBufferDesc,
            &indexBufferData,
            indexBuffer
        )
    );

    SetDebugName(*vertexBuffer, Platform::String::Concat(debugName,
        "_VertexBuffer"));
    SetDebugName(*indexBuffer, Platform::String::Concat(debugName,
        "_IndexBuffer"));

    if (vertexCount != nullptr)
    {
        *vertexCount = numVertices;
    }
    if (indexCount != nullptr)
    {
        *indexCount = numIndices;
    }
}

void BasicLoader::LoadTexture(
    _In_ Platform::String^ filename,
    _Out_opt_ ID3D11Texture2D** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView
)
{
    Platform::Array<byte>^ textureData = m_basicReaderWriter-
>ReadData(filename);

    CreateTexture(
        GetExtension(filename) == "dds",
        textureData->Data,
        textureData->Length,
        texture,
        textureView,
        filename
    );
}

task<void> BasicLoader::LoadTextureAsync(
    _In_ Platform::String^ filename,
    _Out_opt_ ID3D11Texture2D** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then(=[](const
Platform::Array<byte>^ textureData)
    {
        CreateTexture(
            GetExtension(filename) == "dds",
            textureData->Data,
            textureData->Length,
            texture,

```

```

        textureView,
        filename
    );
});
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
    layoutDesc[],
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11VertexShader** shader,
    _Out_opt_ ID3D11InputLayout** layout
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter-
    >ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreateVertexShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);

    if (layout != nullptr)
    {
        CreateInputLayout(
            bytecode->Data,
            bytecode->Length,
            layoutDesc,
            layoutDescNumElements,
            layout
        );

        SetDebugName(*layout, filename);
    }
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _In_reads_opt_(layoutDescNumElements) D3D11_INPUT_ELEMENT_DESC
    layoutDesc[],
    _In_ uint32 layoutDescNumElements,
    _Out_ ID3D11VertexShader** shader,
    _Out_opt_ ID3D11InputLayout** layout
)
{
    // This method assumes that the lifetime of input arguments may be
    shorter
    // than the duration of this task. In order to ensure accurate results,

```



```

a
    // copy of all arguments passed by pointer must be made. The method
    then
    // ensures that the lifetime of the copied data exceeds that of the
    task.

    // Create copies of the layoutDesc array as well as the SemanticName
    strings,
    // both of which are pointers to data whose lifetimes may be shorter
    than that
    // of this method's task.
    shared_ptr<vector<D3D11_INPUT_ELEMENT_DESC>> layoutDescCopy;
    shared_ptr<vector<string>> layoutDescSemanticNamesCopy;
    if (layoutDesc != nullptr)
    {
        layoutDescCopy.reset(
            new vector<D3D11_INPUT_ELEMENT_DESC>(
                layoutDesc,
                layoutDesc + layoutDescNumElements
            )
        );

        layoutDescSemanticNamesCopy.reset(
            new vector<string>(layoutDescNumElements)
        );

        for (uint32 i = 0; i < layoutDescNumElements; i++)
        {
            layoutDescSemanticNamesCopy->
            at(i).assign(layoutDesc[i].SemanticName);
        }
    }

    return m_basicReaderWriter->ReadDataAsync(filename).then([=])(const
    Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateVertexShader(
                bytecode->Data,
                bytecode->Length,
                nullptr,
                shader
            )
        );

        SetDebugName(*shader, filename);

        if (layout != nullptr)
        {
            if (layoutDesc != nullptr)
            {
                // Reassign the SemanticName elements of the layoutDesc
                array copy to point
                // to the corresponding copied strings. Performing the
                assignment inside the

```

```

        // lambda body ensures that the lambda will take a reference
        to the shared_ptr
        // that holds the data. This will guarantee that the data
        is still valid when
        // CreateInputLayout is called.
        for (uint32 i = 0; i < layoutDescNumElements; i++)
        {
            layoutDescCopy->at(i).SemanticName =
layoutDescSemanticNamesCopy->at(i).c_str();
        }
    }

    CreateInputLayout(
        bytecode->Data,
        bytecode->Length,
        layoutDesc == nullptr ? nullptr : layoutDescCopy->data(),
        layoutDescNumElements,
        layout
    );

    SetDebugName(*layout, filename);
}
});
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _Out_ ID3D11PixelShader** shader
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter-
>ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreatePixelShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11PixelShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreatePixelShader(

```

```

        bytecode->Data,
        bytecode->Length,
        nullptr,
        shader
    )
};

    SetDebugName(*shader, filename);
});
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _Out_ ID3D11ComputeShader** shader
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter-
>ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreateComputeShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11ComputeShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=])(const
Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateComputeShader(
                bytecode->Data,
                bytecode->Length,
                nullptr,
                shader
            )
        );

        SetDebugName(*shader, filename);
    });
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _Out_ ID3D11GeometryShader** shader

```

```

    )
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter-
>ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreateGeometryShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11GeometryShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=](const
Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateGeometryShader(
                bytecode->Data,
                bytecode->Length,
                nullptr,
                shader
            )
        );

        SetDebugName(*shader, filename);
    });
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _In_reads_opt_(numEntries) const D3D11_SO_DECLARATION_ENTRY*
streamOutDeclaration,
    _In_ uint32 numEntries,
    _In_reads_opt_(numStrides) const uint32* bufferStrides,
    _In_ uint32 numStrides,
    _In_ uint32 rasterizedStream,
    _Out_ ID3D11GeometryShader** shader
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter-
>ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreateGeometryShaderWithStreamOutput(
            bytecode->Data,

```

```

        bytecode->Length,
        streamOutDeclaration,
        numEntries,
        bufferStrides,
        numStrides,
        rasterizedStream,
        nullptr,
        shader
    )
};

SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _In_reads_opt_(numEntries) const D3D11_SO_DECLARATION_ENTRY*
streamOutDeclaration,
    _In_ uint32 numEntries,
    _In_reads_opt_(numStrides) const uint32* bufferStrides,
    _In_ uint32 numStrides,
    _In_ uint32 rasterizedStream,
    _Out_ ID3D11GeometryShader** shader
)
{
    // This method assumes that the lifetime of input arguments may be
shorter
    // than the duration of this task. In order to ensure accurate results,
a
    // copy of all arguments passed by pointer must be made. The method
then
    // ensures that the lifetime of the copied data exceeds that of the
task.

    // Create copies of the streamOutDeclaration array as well as the
SemanticName
    // strings, both of which are pointers to data whose lifetimes may be
shorter
    // than that of this method's task.
    shared_ptr<vector<D3D11_SO_DECLARATION_ENTRY>> streamOutDeclarationCopy;
    shared_ptr<vector<string>> streamOutDeclarationSemanticNamesCopy;
    if (streamOutDeclaration != nullptr)
    {
        streamOutDeclarationCopy.reset(
            new vector<D3D11_SO_DECLARATION_ENTRY>(
                streamOutDeclaration,
                streamOutDeclaration + numEntries
            )
        );

        streamOutDeclarationSemanticNamesCopy.reset(
            new vector<string>(numEntries)
        );

        for (uint32 i = 0; i < numEntries; i++)

```

```

        {
            streamOutDeclarationSemanticNamesCopy->at(i).assign(streamOutDeclaration[i].SemanticName);
        }
    }

    // Create a copy of the bufferStrides array, which is a pointer to data
    // whose lifetime may be shorter than that of this method's task.
    shared_ptr<vector<uint32>> bufferStridesCopy;
    if (bufferStrides != nullptr)
    {
        bufferStridesCopy.reset(
            new vector<uint32>(
                bufferStrides,
                bufferStrides + numStrides
            )
        );
    }

    return m_basicReaderWriter->ReadDataAsync(filename).then(=[](const
Platform::Array<byte>^ bytecode)
    {
        if (streamOutDeclaration != nullptr)
        {
            // Reassign the SemanticName elements of the
streamOutDeclaration array copy to
            // point to the corresponding copied strings. Performing the
assignment inside the
            // lambda body ensures that the lambda will take a reference to
the shared_ptr
            // that holds the data. This will guarantee that the data is
still valid when
            // CreateGeometryShaderWithStreamOutput is called.
            for (uint32 i = 0; i < numEntries; i++)
            {
                streamOutDeclarationCopy->at(i).SemanticName =
streamOutDeclarationSemanticNamesCopy->at(i).c_str();
            }
        }

        DX::ThrowIfFailed(
            m_d3dDevice->CreateGeometryShaderWithStreamOutput(
                bytecode->Data,
                bytecode->Length,
                streamOutDeclaration == nullptr ? nullptr :
streamOutDeclarationCopy->data(),
                numEntries,
                bufferStrides == nullptr ? nullptr : bufferStridesCopy->data(),
                numStrides,
                rasterizedStream,
                nullptr,
                shader
            )
        );
    });

```

```

        SetDebugName(*shader, filename);
    });
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _Out_ ID3D11HullShader** shader
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter->ReadData(filename);

    DX::ThrowIfFailed(
        m_d3dDevice->CreateHullShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11HullShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then(=[](const
Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateHullShader(
                bytecode->Data,
                bytecode->Length,
                nullptr,
                shader
            )
        );

        SetDebugName(*shader, filename);
    });
}

void BasicLoader::LoadShader(
    _In_ Platform::String^ filename,
    _Out_ ID3D11DomainShader** shader
)
{
    Platform::Array<byte>^ bytecode = m_basicReaderWriter->ReadData(filename);

    DX::ThrowIfFailed(

```

```

        m_d3dDevice->CreateDomainShader(
            bytecode->Data,
            bytecode->Length,
            nullptr,
            shader
        )
    );

    SetDebugName(*shader, filename);
}

task<void> BasicLoader::LoadShaderAsync(
    _In_ Platform::String^ filename,
    _Out_ ID3D11DomainShader** shader
)
{
    return m_basicReaderWriter->ReadDataAsync(filename).then([=])(const
Platform::Array<byte>^ bytecode)
    {
        DX::ThrowIfFailed(
            m_d3dDevice->CreateDomainShader(
                bytecode->Data,
                bytecode->Length,
                nullptr,
                shader
            )
        );

        SetDebugName(*shader, filename);
    });
}

void BasicLoader::LoadMesh(
    _In_ Platform::String^ filename,
    _Out_ ID3D11Buffer** vertexBuffer,
    _Out_ ID3D11Buffer** indexBuffer,
    _Out_opt_ uint32* vertexCount,
    _Out_opt_ uint32* indexCount
)
{
    Platform::Array<byte>^ meshData = m_basicReaderWriter-
>ReadData(filename);

    CreateMesh(
        meshData->Data,
        vertexBuffer,
        indexBuffer,
        vertexCount,
        indexCount,
        filename
    );
}

task<void> BasicLoader::LoadMeshAsync(
    _In_ Platform::String^ filename,

```



```

        _Out_ ID3D11Buffer** vertexBuffer,
        _Out_ ID3D11Buffer** indexBuffer,
        _Out_opt_ uint32* vertexCount,
        _Out_opt_ uint32* indexCount
    )
{
    return m_basicReaderWriter->ReadDataAsync(filename).then(=[])(const
Platform::Array<byte>^ meshData)
    {
        CreateMesh(
            meshData->Data,
            vertexBuffer,
            indexBuffer,
            vertexCount,
            indexCount,
            filename
        );
    });
}

```

Complete code for BasicReaderWriter

Article • 10/20/2022

Complete code for a class and methods for reading and writing binary data files in general. Used by the [BasicLoader](#) class.

This topic contains these sections:

- [Technologies](#)
- [Requirements](#)
- [View the code \(C++\)](#)

Download location

This sample is not available for download.

Technologies

Programming languages - C++

Programming models - Windows Runtime

Requirements

Minimum supported client - Windows 10

Minimum supported server - Windows Server 2016 Technical Preview

View the code (C++)

BasicReaderWriter.h

C++

```
//// THIS CODE AND INFORMATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF
//// ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO
//// THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR FITNESS FOR A
//// PARTICULAR PURPOSE.
////
//// Copyright (c) Microsoft Corporation. All rights reserved

#pragma once
```

```

#include <ppltasks.h>

// A simple reader/writer class that provides support for reading and
// writing
// files on disk. Provides synchronous and asynchronous methods.
ref class BasicReaderWriter
{
private:
    Windows::Storage::StorageFolder^ m_location;
    Platform::String^ m_locationPath;

internal:
    BasicReaderWriter();
    BasicReaderWriter(
        _In_ Windows::Storage::StorageFolder^ folder
    );

    Platform::Array<byte>^ ReadData(
        _In_ Platform::String^ filename
    );

    concurrency::task<Platform::Array<byte>^> ReadDataAsync(
        _In_ Platform::String^ filename
    );

    uint32 WriteData(
        _In_ Platform::String^ filename,
        _In_ const Platform::Array<byte>^ fileData
    );

    concurrency::task<void> WriteDataAsync(
        _In_ Platform::String^ filename,
        _In_ const Platform::Array<byte>^ fileData
    );
};

```

BasicReaderWriter.cpp

C++

```

using namespace Microsoft::WRL;
using namespace Windows::Storage;
using namespace Windows::Storage::FileProperties;
using namespace Windows::Storage::Streams;
using namespace Windows::Foundation;
using namespace Windows::ApplicationModel;
using namespace concurrency;

BasicReaderWriter::BasicReaderWriter()
{
    m_location = Package::Current->InstalledLocation;
    m_locationPath = Platform::String::Concat(m_location->Path, "\\");
}

```

```

}

BasicReaderWriter::BasicReaderWriter(
    _In_ Windows::Storage::StorageFolder^ folder
)
{
    m_location = folder;
    Platform::String^ path = m_location->Path;
    if (path->Length() == 0)
    {
        // Applications are not permitted to access certain
        // folders, such as the Documents folder, using this
        // code path. In such cases, the Path property for
        // the folder will be an empty string.
        throw ref new Platform::FailureException();
    }
    m_locationPath = Platform::String::Concat(path, "\\");
}

Platform::Array<byte>^ BasicReaderWriter::ReadData(
    _In_ Platform::String^ filename
)
{
    CREATEFILE2_EXTENDED_PARAMETERS extendedParams = {0};
    extendedParams.dwSize = sizeof(CREATEFILE2_EXTENDED_PARAMETERS);
    extendedParams.dwFileAttributes = FILE_ATTRIBUTE_NORMAL;
    extendedParams.dwFileFlags = FILE_FLAG_SEQUENTIAL_SCAN;
    extendedParams.dwSecurityQosFlags = SECURITY_ANONYMOUS;
    extendedParams.lpSecurityAttributes = nullptr;
    extendedParams.hTemplateFile = nullptr;

    Wrappers::FileHandle file(
        CreateFile2(
            Platform::String::Concat(m_locationPath, filename)->Data(),
            GENERIC_READ,
            FILE_SHARE_READ,
            OPEN_EXISTING,
            &extendedParams
        )
    );
    if (file.Get() == INVALID_HANDLE_VALUE)
    {
        throw ref new Platform::FailureException();
    }

    FILE_STANDARD_INFO fileInfo = {0};
    if (!GetFileInformationByHandleEx(
        file.Get(),
        FileStandardInfo,
        &fileInfo,
        sizeof(fileInfo)
    ))
    {
        throw ref new Platform::FailureException();
    }
}

```

```

        if (fileInfo.EndOfFile.HighPart != 0)
        {
            throw ref new Platform::OutOfMemoryException();
        }

        Platform::Array<byte>^ fileData = ref new Platform::Array<byte>
(fileInfo.EndOfFile.LowPart);

        if (!ReadFile(
            file.Get(),
            fileData->Data,
            fileData->Length,
            nullptr,
            nullptr
        ))
        {
            throw ref new Platform::FailureException();
        }

        return fileData;
    }

task<Platform::Array<byte>^> BasicReaderWriter::ReadDataAsync(
    _In_ Platform::String^ filename
)
{
    return task<StorageFile^>(m_location->GetFileAsync(filename)).then([=]
(StorageFile^ file)
    {
        return FileIO::ReadBufferAsync(file);
    }).then([=](IBuffer^ buffer)
    {
        auto fileData = ref new Platform::Array<byte>(buffer->Length);
        DataReader::FromBuffer(buffer)->ReadBytes(fileData);
        return fileData;
    });
}

uint32 BasicReaderWriter::WriteData(
    _In_ Platform::String^ filename,
    _In_ const Platform::Array<byte>^ fileData
)
{
    CREATEFILE2_EXTENDED_PARAMETERS extendedParams = {0};
    extendedParams.dwSize = sizeof(CREATEFILE2_EXTENDED_PARAMETERS);
    extendedParams.dwFileAttributes = FILE_ATTRIBUTE_NORMAL;
    extendedParams.dwFileFlags = FILE_FLAG_SEQUENTIAL_SCAN;
    extendedParams.dwSecurityQosFlags = SECURITY_ANONYMOUS;
    extendedParams.lpSecurityAttributes = nullptr;
    extendedParams.hTemplateFile = nullptr;

    Wrappers::FileHandle file(
        CreateFile2(
            Platform::String::Concat(m_locationPath, filename)->Data(),

```

```

        GENERIC_WRITE,
        0,
        CREATE_ALWAYS,
        &extendedParams
    )
);
if (file.Get() == INVALID_HANDLE_VALUE)
{
    throw ref new Platform::FailureException();
}

DWORD numBytesWritten;
if (
    !WriteFile(
        file.Get(),
        fileData->Data,
        fileData->Length,
        &numBytesWritten,
        nullptr
    ) ||
    numBytesWritten != fileData->Length
)
{
    throw ref new Platform::FailureException();
}

return numBytesWritten;
}

task<void> BasicReaderWriter::WriteDataAsync(
    _In_ Platform::String^ filename,
    _In_ const Platform::Array<byte>^ fileData
)
{
    return task<StorageFile^>(m_location->CreateFileAsync(filename,
        CreationCollisionOption::ReplaceExisting)).then([=](StorageFile^ file)
    {
        FileIO::WriteBytesAsync(file, fileData);
    });
}

```

Complete code for DDSTextureLoader

Article • 10/20/2022

Complete code for a class and method that loads a DDS texture from memory. See [Load resources in your DirectX game](#) for a discussion about textures and using this example.

This topic contains these sections:

- [Technologies](#)
- [Requirements](#)
- [View the code \(C++\)](#)

Download location

The DDSTextureLoader can be found on GitHub in [DirectXTex](#) and *DirectX Tool Kit* for [DX11](#) / [DX12](#).

Technologies

Programming languages - C++

Programming models - Windows Runtime

Requirements

Minimum supported client - Windows 10

Minimum supported server - Windows Server 2016 Technical Preview

View the code (C++)

DDSTextureLoader.h

C++

```
//-----  
-----  
// File: DDSTextureLoader.h  
//  
// Function for loading a DDS texture and creating a Direct3D 11 runtime  
// resource for it  
//
```

```

// Note this function is useful as a light-weight runtime loader for DDS
files. For
// a full-featured DDS file reader, writer, and texture processing pipeline,
see
// the 'Texconv' sample and the 'DirectXTex' library.
//
// Copyright (c) Microsoft Corporation.
// Licensed under the MIT License.
//-----
-----
#pragma once

void CreateDDSTextureFromMemory(
    _In_ ID3D11Device* d3dDevice,
    _In_reads_bytes_(ddsDataSize) const byte* ddsData,
    _In_ size_t ddsDataSize,
    _Out_opt_ ID3D11Resource** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView,
    _In_ size_t maxsize = 0
);

```

DDSTextureLoader.cpp

C++

```

//-----
-----
// File: DDSTextureLoader.cpp
//
// Function for loading a DDS texture and creating a Direct3D 11 runtime
resource for it
//
// Note this function is useful as a light-weight runtime loader for DDS
files. For
// a full-featured DDS file reader, writer, and texture processing pipeline,
see
// the 'Texconv' sample and the 'DirectXTex' library.
//
// Copyright (c) Microsoft Corporation.
// Licensed under the MIT License.
//-----
-----
#include "pch.h"
#include <dxgiformat.h>
#include <assert.h>
#include <memory>
#include <algorithm>
#include "DDSTextureLoader.h"
#include "DirectXSample.h"

using namespace Microsoft::WRL;

```



```

//-----
// Macros
//-----
-----
#ifndef MAKEFOURCC
    #define MAKEFOURCC(ch0, ch1, ch2, ch3)          \
        ((uint32)(byte)(ch0) | ((uint32)(byte)(ch1) << 8) |      \
         ((uint32)(byte)(ch2) << 16) | ((uint32)(byte)(ch3) << 24))
#endif /* defined(MAKEFOURCC) */

//-----
// DDS file structure definitions
//
// See DDS.h in the 'Texconv' sample and the 'DirectXTex' library.
//-----
-----
#pragma pack(push, 1)

#define DDS_MAGIC 0x20534444 // "DDS "

struct DDS_PIXELFORMAT
{
    uint32    size;
    uint32    flags;
    uint32    fourCC;
    uint32    RGBBitCount;
    uint32    RBitMask;
    uint32    GBitMask;
    uint32    BBitMask;
    uint32    ABitMask;
};

#define DDS_FOURCC          0x00000004 // DDPF_FOURCC
#define DDS_RGB             0x00000040 // DDPF_RGB
#define DDS_RGBA            0x00000041 // DDPF_RGB | DDPF_ALPHAPIXELS
#define DDS_LUMINANCE       0x00020000 // DDPF_LUMINANCE
#define DDS_LUMINANCEA      0x00020001 // DDPF_LUMINANCE | DDPF_ALPHAPIXELS
#define DDS_ALPHA           0x00000002 // DDPF_ALPHA
#define DDS_PALETTE8        0x00000020 // DDPF_PALETTEINDEXED8

#define DDS_HEADER_FLAGS_TEXTURE      0x00001007 // DDSD_CAPS |
    DDSD_HEIGHT | DDSD_WIDTH | DDSD_PIXELFORMAT
#define DDS_HEADER_FLAGS_MIPMAP       0x00020000 // DDSD_MIPMAPCOUNT
#define DDS_HEADER_FLAGS_VOLUME       0x00800000 // DDSD_DEPTH
#define DDS_HEADER_FLAGS_PITCH        0x00000008 // DDSD_PITCH
#define DDS_HEADER_FLAGS_LINEARSIZE   0x00080000 // DDSD_LINEARSIZE

#define DDS_HEIGHT 0x00000002 // DDSD_HEIGHT
#define DDS_WIDTH  0x00000004 // DDSD_WIDTH

#define DDS_SURFACE_FLAGS_TEXTURE 0x00001000 // DDSCAPS_TEXTURE
#define DDS_SURFACE_FLAGS_MIPMAP  0x00400008 // DDSCAPS_COMPLEX |
    DDSCAPS_MIPMAP

```

```

#define DDS_SURFACE_FLAGS_CUBEMAP 0x00000008 // DDSCAPS_COMPLEX

#define DDS_CUBEMAP_POSITIVEX 0x00000600 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_POSITIVEX
#define DDS_CUBEMAP_NEGATIVEX 0x00000a00 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_NEGATIVEX
#define DDS_CUBEMAP_POSITIVEY 0x00001200 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_POSITIVEY
#define DDS_CUBEMAP_NEGATIVEY 0x00002200 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_NEGATIVEY
#define DDS_CUBEMAP_POSITIVEZ 0x00004200 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_POSITIVEZ
#define DDS_CUBEMAP_NEGATIVEZ 0x00008200 // DDSCAPS2_CUBEMAP |
DDSCAPS2_CUBEMAP_NEGATIVEZ

#define DDS_CUBEMAP_ALLFACES (DDS_CUBEMAP_POSITIVEX | DDS_CUBEMAP_NEGATIVEX
| \
                                DDS_CUBEMAP_POSITIVEY | DDS_CUBEMAP_NEGATIVEY
| \
                                DDS_CUBEMAP_POSITIVEZ | DDS_CUBEMAP_NEGATIVEZ)

#define DDS_CUBEMAP 0x00000200 // DDSCAPS2_CUBEMAP

#define DDS_FLAGS_VOLUME 0x00200000 // DDSCAPS2_VOLUME

typedef struct
{
    uint32          size;
    uint32          flags;
    uint32          height;
    uint32          width;
    uint32          pitchOrLinearSize;
    uint32          depth; // only if DDS_HEADER_FLAGS_VOLUME is set in
flags
    uint32          mipMapCount;
    uint32          reserved1[11];
    DDS_PIXELFORMAT ddspf;
    uint32          caps;
    uint32          caps2;
    uint32          caps3;
    uint32          caps4;
    uint32          reserved2;
} DDS_HEADER;

typedef struct
{
    DXGI_FORMAT dxgiFormat;
    uint32      resourceDimension;
    uint32      miscFlag; // See D3D11_RESOURCE_MISC_FLAG
    uint32      arraySize;
    uint32      reserved;
} DDS_HEADER_DXT10;

#pragma pack(pop)

```

```

//-----
// Return the BPP for a particular format.
//-----
static size_t BitsPerPixel(_In_ DXGI_FORMAT fmt)
{
    switch (fmt)
    {
        case DXGI_FORMAT_R32G32B32A32_TYPELESS:
        case DXGI_FORMAT_R32G32B32A32_FLOAT:
        case DXGI_FORMAT_R32G32B32A32_UINT:
        case DXGI_FORMAT_R32G32B32A32_SINT:
            return 128;

        case DXGI_FORMAT_R32G32B32_TYPELESS:
        case DXGI_FORMAT_R32G32B32_FLOAT:
        case DXGI_FORMAT_R32G32B32_UINT:
        case DXGI_FORMAT_R32G32B32_SINT:
            return 96;

        case DXGI_FORMAT_R16G16B16A16_TYPELESS:
        case DXGI_FORMAT_R16G16B16A16_FLOAT:
        case DXGI_FORMAT_R16G16B16A16_UNORM:
        case DXGI_FORMAT_R16G16B16A16_UINT:
        case DXGI_FORMAT_R16G16B16A16_SNORM:
        case DXGI_FORMAT_R16G16B16A16_SINT:
        case DXGI_FORMAT_R32G32_TYPELESS:
        case DXGI_FORMAT_R32G32_FLOAT:
        case DXGI_FORMAT_R32G32_UINT:
        case DXGI_FORMAT_R32G32_SINT:
        case DXGI_FORMAT_R32G8X24_TYPELESS:
        case DXGI_FORMAT_D32_FLOAT_S8X24_UINT:
        case DXGI_FORMAT_R32_FLOAT_X8X24_TYPELESS:
        case DXGI_FORMAT_X32_TYPELESS_G8X24_UINT:
            return 64;

        case DXGI_FORMAT_R10G10B10A2_TYPELESS:
        case DXGI_FORMAT_R10G10B10A2_UNORM:
        case DXGI_FORMAT_R10G10B10A2_UINT:
        case DXGI_FORMAT_R11G11B10_FLOAT:
        case DXGI_FORMAT_R8G8B8A8_TYPELESS:
        case DXGI_FORMAT_R8G8B8A8_UNORM:
        case DXGI_FORMAT_R8G8B8A8_UNORM_SRGB:
        case DXGI_FORMAT_R8G8B8A8_UINT:
        case DXGI_FORMAT_R8G8B8A8_SNORM:
        case DXGI_FORMAT_R8G8B8A8_SINT:
        case DXGI_FORMAT_R16G16_TYPELESS:
        case DXGI_FORMAT_R16G16_FLOAT:
        case DXGI_FORMAT_R16G16_UNORM:
        case DXGI_FORMAT_R16G16_UINT:
        case DXGI_FORMAT_R16G16_SNORM:
        case DXGI_FORMAT_R16G16_SINT:
        case DXGI_FORMAT_R32_TYPELESS:
        case DXGI_FORMAT_D32_FLOAT:
    }
}

```

```
case DXGI_FORMAT_R32_FLOAT:
case DXGI_FORMAT_R32_UINT:
case DXGI_FORMAT_R32_SINT:
case DXGI_FORMAT_R24G8_TYPELESS:
case DXGI_FORMAT_D24_UNORM_S8_UINT:
case DXGI_FORMAT_R24_UNORM_X8_TYPELESS:
case DXGI_FORMAT_X24_TYPELESS_G8_UINT:
case DXGI_FORMAT_R9G9B9E5_SHAREDEXP:
case DXGI_FORMAT_R8G8_B8G8_UNORM:
case DXGI_FORMAT_G8R8_G8B8_UNORM:
case DXGI_FORMAT_B8G8R8A8_UNORM:
case DXGI_FORMAT_B8G8R8X8_UNORM:
case DXGI_FORMAT_R10G10B10_XR_BIAS_A2_UNORM:
case DXGI_FORMAT_B8G8R8A8_TYPELESS:
case DXGI_FORMAT_B8G8R8A8_UNORM_SRGB:
case DXGI_FORMAT_B8G8R8X8_TYPELESS:
case DXGI_FORMAT_B8G8R8X8_UNORM_SRGB:
    return 32;
```

```
case DXGI_FORMAT_R8G8_TYPELESS:
case DXGI_FORMAT_R8G8_UNORM:
case DXGI_FORMAT_R8G8_UINT:
case DXGI_FORMAT_R8G8_SNORM:
case DXGI_FORMAT_R8G8_SINT:
case DXGI_FORMAT_R16_TYPELESS:
case DXGI_FORMAT_R16_FLOAT:
case DXGI_FORMAT_D16_UNORM:
case DXGI_FORMAT_R16_UNORM:
case DXGI_FORMAT_R16_UINT:
case DXGI_FORMAT_R16_SNORM:
case DXGI_FORMAT_R16_SINT:
case DXGI_FORMAT_B5G6R5_UNORM:
case DXGI_FORMAT_B5G5R5A1_UNORM:
case DXGI_FORMAT_B4G4R4A4_UNORM:
    return 16;
```

```
case DXGI_FORMAT_R8_TYPELESS:
case DXGI_FORMAT_R8_UNORM:
case DXGI_FORMAT_R8_UINT:
case DXGI_FORMAT_R8_SNORM:
case DXGI_FORMAT_R8_SINT:
case DXGI_FORMAT_A8_UNORM:
    return 8;
```

```
case DXGI_FORMAT_R1_UNORM:
    return 1;
```

```
case DXGI_FORMAT_BC1_TYPELESS:
case DXGI_FORMAT_BC1_UNORM:
case DXGI_FORMAT_BC1_UNORM_SRGB:
case DXGI_FORMAT_BC4_TYPELESS:
case DXGI_FORMAT_BC4_UNORM:
case DXGI_FORMAT_BC4_SNORM:
    return 4;
```

```

        case DXGI_FORMAT_BC2_TYPELESS:
        case DXGI_FORMAT_BC2_UNORM:
        case DXGI_FORMAT_BC2_UNORM_SRGB:
        case DXGI_FORMAT_BC3_TYPELESS:
        case DXGI_FORMAT_BC3_UNORM:
        case DXGI_FORMAT_BC3_UNORM_SRGB:
        case DXGI_FORMAT_BC5_TYPELESS:
        case DXGI_FORMAT_BC5_UNORM:
        case DXGI_FORMAT_BC5_SNORM:
        case DXGI_FORMAT_BC6H_TYPELESS:
        case DXGI_FORMAT_BC6H_UF16:
        case DXGI_FORMAT_BC6H_SF16:
        case DXGI_FORMAT_BC7_TYPELESS:
        case DXGI_FORMAT_BC7_UNORM:
        case DXGI_FORMAT_BC7_UNORM_SRGB:
            return 8;

    default:
        return 0;
}

}

//-----
// Get surface information for a particular format.
//-----

static void GetSurfaceInfo(
    _In_ size_t width,
    _In_ size_t height,
    _In_ DXGI_FORMAT fmt,
    _Out_opt_ size_t* outNumBytes,
    _Out_opt_ size_t* outRowBytes,
    _Out_opt_ size_t* outNumRows
)
{
    size_t numBytes = 0;
    size_t rowBytes = 0;
    size_t numRows = 0;

    bool bc = false;
    bool packed = false;
    size_t bcnBytesPerBlock = 0;
    switch (fmt)
    {
        case DXGI_FORMAT_BC1_TYPELESS:
        case DXGI_FORMAT_BC1_UNORM:
        case DXGI_FORMAT_BC1_UNORM_SRGB:
        case DXGI_FORMAT_BC4_TYPELESS:
        case DXGI_FORMAT_BC4_UNORM:
        case DXGI_FORMAT_BC4_SNORM:
            bc = true;
            bcnBytesPerBlock = 8;
            break;
    }

```